EECE 7205: Fundamentals of Engineering Algorithms
Fall 2026
Binary Trees Assignment

Paulina Skurzak

Question 1: Maximum depth of a binary tree

Input: arr = {5, 12, 8, 21, 3, 17, 9}
Output: number of edges on the longest path from the root to a leaf
(a) Pseudocode:

```
MAX-DEPTH(node)
Input: node
Output: integer depth
Initial call: MAX-DEPTH(root)
1. if node = NULL
2.     return -1
3. leftDepth <- MAX-DEPTH(node.left)
4. rightDepth <- MAX-DEPTH(node.right)
5. return 1 + max(leftDepth, rightDepth)
```

(b) Worked Example:

Call	Returns
MAX-DEPTH(21.left) = MAX-DEPTH(NULL)	-1
MAX-DEPTH(21.right) = MAX-DEPTH(NULL)	-1
MAX-DEPTH(21)	0
MAX-DEPTH(3.left) = MAX-DEPTH(NULL)	-1
MAX-DEPTH(3.right) = MAX-DEPTH(NULL)	-1
MAX-DEPTH(3)	0
MAX-DEPTH(12)	1
MAX-DEPTH(17.left) = MAX-DEPTH(NULL)	-1
MAX-DEPTH(17.right) = MAX-DEPTH(NULL)	-1
MAX-DEPTH(17)	0
MAX-DEPTH(9.left) = MAX-DEPTH(NULL)	-1
MAX-DEPTH(9.right) = MAX-DEPTH(NULL)	-1
MAX-DEPTH(9)	0
MAX-DEPTH(8)	1
MAX-DEPTH(5)	2

I got 2 at the end, which is what the problem wanted, so I think the pseudocode works.

(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
1	if node=NULL	1	$2n+1$	15
2	return -1	1	$n+1$	8
3	leftDepth $\leftarrow$ MAX-DEPTH(node.left)	1	n	7
4	rightDepth $\leftarrow$ MAX-DEPTH(node.right)	1	n	7
5	return $1 + \max(\cdot, \cdot)$	1	n	7

$T(n) = (2n+1) + (n+1) + n + n + n = 6n + 2 \rightarrow \Theta(n)$
$15 + 8 + 7 + 7 + 7 = 44$, $6(7) + 2 = 44$ which matches
(d) Final Complexity:

Both best and worst case end up being $\Theta(n)$ here. I don't think there's a way to skip any nodes with this one, since you have to check every single node and every empty spot to know the real answer. Extra space is $O(h)$ because of how recursion works, it only keeps track of one path down the tree at a time, not the whole thing.

Twist: Would your frequency count change for a tree that is a single chain of 7 nodes? Explain.

I think the frequency count stays the same even for a chain. The formulas are all just based on n, they don't really care what the tree looks like. But the space thing is different I think, since a chain is way taller, height 6 instead of 2, so the recursion has to stack up a lot more calls at once, closer to $O(n)$ instead of $O(2)$.

Question 2: Sum of all nodes

Input: the tree from arr
Expected output: 75
(a) Pseudocode:

```
SUM-NODES(node)
Input: node
Output: integer sum
Initial call: SUM-NODES(root)
1. if node = NULL
2.     return 0
3. leftSum <- SUM-NODES(node.left)
4. rightSum <- SUM-NODES(node.right)
5. return node.val + leftSum + rightSum
```

(b) Worked Example:

Call	Returns
SUM-NODES(21.left)=SUM-NODES(NULL)	0
SUM-NODES(21.right)=SUM-NODES(NULL)	0
SUM-NODES(21)	21
SUM-NODES(3.left)=SUM-NODES(NULL)	0
SUM-NODES(3.right)=SUM-NODES(NULL)	0
SUM-NODES(3)	3
SUM-NODES(12)	36
SUM-NODES(17.left)=SUM-NODES(NULL)	0
SUM-NODES(17.right)=SUM-NODES(NULL)	0
SUM-NODES(17)	17
SUM-NODES(9.left)=SUM-NODES(NULL)	0
SUM-NODES(9.right)=SUM-NODES(NULL)	0
SUM-NODES(9)	9
SUM-NODES(8)	34
SUM-NODES(5)	75

Final output is 75 which matches the expected output.
(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
1	if node=NULL	1	$2n+1$	15
2	return 0	1	$n+1$	8
3	leftSum $\leftarrow$ SUM-NODES(node.left)	1	n	7
4	rightSum $\leftarrow$ SUM-NODES(node.right)	1	n	7
5	return node.val+leftSum+rightSum	1	n	7

$T(n) = (2n+1) + (n+1) + n + n + n = 6n + 2 \rightarrow \Theta(n)$
(d) Final Complexity:
Same as question 1, both cases are $\Theta(n)$ and space is $O(h)$.
Twist: Write it once recursively and once with a queue (level order). Compare the two frequency counts and the extra space each needs.

Queue-based pseudocode:

```
SUM-NODES-QUEUE(root)
1. queue <- [root]
2. total <- 0
3. while queue is not empty
4.      node <- remove front of queue
5.      total <- total + node.val
6.      if node.left != NULL
7.          add node.left to back of queue
8.      if node.right != NULL
9.          add node.right to back of queue
10. return total
```

Queue-based frequency count:

Line	Operation	Cost	Frequency	Count
1	queue $\leftarrow$ [root]	1	1	1
2	total $\leftarrow$ 0	1	1	1
3	while test	1	$n+1$	8
4	remove front	1	n	7
5	total $\leftarrow$ total+node.val	1	n	7
6	check left	1	n	7
7+9	add child	1	$n-1$	6
8	check right	1	n	7
10	return total	1	1	1

$T(n) = 2 + (n+1) + n + n + n + (n-1) + n + 1 \rightarrow \Theta(n)$

Comparison: I did both versions. Time-wise they're both n. But the space is different, recursive only needs $O(h)$ since it's just following one path down at a time. The queue one needs $O(w)$ instead, basically how wide the tree gets. Here h is 2 but w is 4, so recursive uses less space for this specific tree. I'd guess for a really wide tree the queue way would need a lot more space, and for something more like a chain, maybe the queue way is actually better.

Question 3: Count the nodes that have exactly one child

Input: arr after INSERT(root, 4)
Expected output: 1 (node 21, whose only child is 4)
(a) Pseudocode:

```
COUNT-ONE-CHILD(node)
Input: node
Output: integer count (number of nodes with exactly one
    child)
Initial call: COUNT-ONE-CHILD(root)
1. if node = NULL
2.     return 0
3. leftCount <- COUNT-ONE-CHILD(node.left)
4. rightCount <- COUNT-ONE-CHILD(node.right)
5. hasLeft <- (node.left != NULL)
6. hasRight <- (node.right != NULL)
7. if (hasLeft = TRUE and hasRight = FALSE) or (hasLeft =
    FALSE and hasRight = TRUE)
8.     return leftCount + rightCount + 1
9. else
10.    return leftCount + rightCount
```

(b) Worked Example:

Call	Lines executed	Returns
COUNT-ONE-CHILD(4.left)=COUNT-ONE-CHILD(NULL)	1,2	0
COUNT-ONE-CHILD(4.right)=COUNT-ONE-CHILD(NULL)	1,2	0
COUNT-ONE-CHILD(4)	1,3,4,5,6,7,9,10	0
COUNT-ONE-CHILD(21.right)=COUNT-ONE-CHILD(NULL)	1,2	0
COUNT-ONE-CHILD(21)	1,3,4,5,6,7,8	1
COUNT-ONE-CHILD(3.left)=COUNT-ONE-CHILD(NULL)	1,2	0
COUNT-ONE-CHILD(3.right)=COUNT-ONE-CHILD(NULL)	1,2	0
COUNT-ONE-CHILD(3)	1,3,4,5,6,7,9,10	0
COUNT-ONE-CHILD(12)	1,3,4,5,6,7,9,10	1
COUNT-ONE-CHILD(17.left)=COUNT-ONE-CHILD(NULL)	1,2	0
COUNT-ONE-CHILD(17.right)=COUNT-ONE-CHILD(NULL)	1,2	0
COUNT-ONE-CHILD(17)	1,3,4,5,6,7,9,10	0
COUNT-ONE-CHILD(9.left)=COUNT-ONE-CHILD(NULL)	1,2	0
COUNT-ONE-CHILD(9.right)=COUNT-ONE-CHILD(NULL)	1,2	0
COUNT-ONE-CHILD(9)	1,3,4,5,6,7,9,10	0
COUNT-ONE-CHILD(8)	1,3,4,5,6,7,9,10	0
COUNT-ONE-CHILD(5)	1,3,4,5,6,7,9,10	1

Final output is 1 which matches the expected output.
(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
1	if node=NULL	1	$2n+1$	17
2	return 0	1	$n+1$	9
3	leftCount ← COUNT-ONE-CHILD(node.left)	1	n	8
4	rightCount ← COUNT-ONE-CHILD(node.right)	1	n	8
5	hasLeft ← (node.left != NULL)	1	n	8
6	hasRight ← (node.right != NULL)	1	n	8
7	if (exactly one child)	1	n	8
8	return leftCount+rightCount+1	1	depends on tree	1
9	else	1	depends on tree	7
10	return leftCount+rightCount	1	depends on tree	7

$T(n) = (2n+1) + (n+1) + n + n + n + n + n + 1 + 7 + 7 \to \Theta(n)$
$17 + 9 + 8 + 8 + 8 + 8 + 8 + 1 + 7 + 7 = 81$

(d) Final Complexity:

Best case: $\Theta(n)$ Worst case: $\Theta(n)$ Extra space: $O(h)$

Twist: Explain why a tree built from an array of n elements has exactly one such node when n is even and none when n is odd.

The last node added always goes into position $n-1$. If n is even, that position turns out to be odd, which means it's a left child, and whatever node it attaches to was a leaf before, so now it only has one kid. But if n is odd, position $n-1$ is even, so the new node becomes a right child instead, and its parent already got a left child earlier, so now it has both. That's why even n gives you one of these nodes and odd n gives you zero.

Question 4: Are two trees identical?

Input: tree A from arr; tree B from arr; tree C from {5, 12, 8, 21, 3, 17, 10}
Expected output: IDENTICAL(A, B) = true; IDENTICAL(A, C) = false
(a) Pseudocode:

```
IDENTICAL(nodeA, nodeB)
Input: nodeA, nodeB
Output: boolean
Initial call: IDENTICAL(rootA, rootB)
1. if nodeA = NULL and nodeB = NULL
2.     return TRUE
3. if nodeA = NULL or nodeB = NULL
4.     return FALSE
5. if nodeA.val != nodeB.val
6.     return FALSE
7. return IDENTICAL(nodeA.left, nodeB.left) and IDENTICAL(
    nodeA.right, nodeB.right)
```

(b) Worked Example:
For A and B every single node matches so it just returns true all the way up. For A and C, everything is the same except one number, 9 vs 10. The code finds this difference right when it compares those two, returns false, and that false just keeps getting passed up through 8 and then 5.

Call	Returns
IDENTICAL(21,21) subtree (all matches)	TRUE
IDENTICAL(3,3) subtree (all matches)	TRUE
IDENTICAL(12,12)	TRUE
IDENTICAL(17,17) subtree (all matches)	TRUE
IDENTICAL(9,10)	FALSE
IDENTICAL(8,8)	FALSE
IDENTICAL(5,5)	FALSE

Final output is FALSE, matching the expected output.
(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
1	both-NULL test	1	$2n+1$	15
2	return TRUE	1	$n+1$	8
3	one-NULL test	1	n	7
4	return FALSE	1	0	0
5	value test	1	n	7
6	return FALSE	1	$\leq n$	1
7	recursive combine	1	n	6

$T(n) = (2n+1) + (n+1) + n + 0 + n + 1 + 6 \rightarrow \Theta(n)$
$15 + 8 + 7 + 0 + 7 + 1 + 6 = 44$
(d) Final Complexity:
Best case here is actually $\Theta(1)$, if the very first thing you check doesn't match, you're done immediately, you don't need to check anything else. Worst case

is $\Theta(n)$ though, like if the trees are actually the same, or the one difference is buried somewhere deep, you'd have to check basically everything before you could say for sure.
Twist: For IDENTICAL(A, C), at which node does your algorithm first find a difference? What is the best case and the worst case for two trees of n nodes?

The difference shows up when comparing 9 and 10. Best case is $\Theta(1)$ since you could get lucky and catch a difference right away. Worst case is $\Theta(n)$ since you might have to check every node before you're sure.

Question 5: Minimum value in a binary tree

Input: the tree from arr
Expected output: 3
(a) Pseudocode:

```
MIN-VALUE(node)
Input: node
Output: integer minimum value
Initial call: MIN-VALUE(root)
1. if node = NULL
2.     return infinity
3. leftMin <- MIN-VALUE(node.left)
4. rightMin <- MIN-VALUE(node.right)
5. return min(node.val, leftMin, rightMin)
```

(b) Worked Example:

Call	Returns
MIN-VALUE(21.left)=MIN-VALUE(NULL)	infinity
MIN-VALUE(21.right)=MIN-VALUE(NULL)	infinity
MIN-VALUE(21)	21
MIN-VALUE(3.left)=MIN-VALUE(NULL)	infinity
MIN-VALUE(3.right)=MIN-VALUE(NULL)	infinity
MIN-VALUE(3)	3
MIN-VALUE(12)	3
MIN-VALUE(17.left)=MIN-VALUE(NULL)	infinity
MIN-VALUE(17.right)=MIN-VALUE(NULL)	infinity
MIN-VALUE(17)	17
MIN-VALUE(9.left)=MIN-VALUE(NULL)	infinity
MIN-VALUE(9.right)=MIN-VALUE(NULL)	infinity
MIN-VALUE(9)	9
MIN-VALUE(8)	8
MIN-VALUE(5)	**3**

Final outputs is 3, matching the expected output.
(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
1	if node=NULL	1	$2n+1$	15
2	return infinity	1	$n+1$	8
3	leftMin $\leftarrow$ MIN-VALUE(node.left)	1	n	7
4	rightMin $\leftarrow$ MIN-VALUE(node.right)	1	n	7
5	return min(node.val,leftMin,rightMin)	1	n	7

$T(n) = (2n+1) + (n+1) + n + n + n = 6n + 2 \rightarrow \Theta(n)$
(d) Final Complexity:
Both cases are $\Theta(n)$ here, I don't think there's any way around that since this isn't a binary search tree, so you really do have to look at every single node, there's no shortcut like always going left.

Twist: How would the algorithm and its complexity change if the tree were a binary search tree?

If it actually was a binary search tree, finding the min would be way easier, you'd just keep going left over and over since the smallest thing is always on the left side. That would make this $\Theta(h)$ instead of $\Theta(n)$, since you're only following one path instead of checking the whole tree.

Question 6: Invert (mirror) a binary tree

Input: the tree from arr
Expected output: level order after inverting: 5, 8, 12, 9, 17, 3, 21
(a) Pseudocode:

```
INVERT(node)
Input: node
Output: node (root of the inverted tree)
Initial call: INVERT(root)
1. if node = NULL
2.     return NULL
3. temp <- node.left
4. node.left <- node.right
5. node.right <- temp
6. INVERT(node.left)
7. INVERT(node.right)
8. return node
```

(b) Worked Example:
Leaves don't really change since swapping NULL with NULL does nothing. For 12 it flips to have 3 on the left and 21 on the right. Same idea for 8. Then the root flips its two whole subtrees.

Final tree, level order: 5, 8, 12, 9, 17, 3, 21, matching the expected output.
(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
1	if node=NULL	1	$2n+1$	15
2	return NULL	1	$n+1$	8
3	temp $\leftarrow$ node.left	1	n	7
4	node.left $\leftarrow$ node.right	1	n	7
5	node.right $\leftarrow$ temp	1	n	7
6	INVERT(node.left)	1	n	7
7	INVERT(node.right)	1	n	7
8	return node	1	n	7

$T(n) = (2n+1) + (n+1) + n + n + n + n + n + n = 9n + 2 \rightarrow \Theta(n)$
(d) Final Complexity:
Best case: $\Theta(n)$. Worst case: $\Theta(n)$, every node must be visited and swapped, no shortcut exists.
Twist: Give the inorder traversal before (21 12 3 5 17 8 9) and after inverting. What do you notice, and why does it happen?

I tried doing inorder on the new tree and got 9 8 17 5 3 12 21, which is exactly the old inorder backwards. I think this happens because inorder checks left then right, but since invert swaps every left and right, doing left-then-right on the new tree is basically the same as doing right-then-left on the old one, so you just get it reversed.

Question 7: Print each level on its own line, and the right side view

Input: the tree from arr
Expected output: levels: [5], [12, 8], [21, 3, 17, 9]; right side view: 5, 8, 9
(a) Pseudocode:

```
PRINT-LEVELS-AND-RIGHT-VIEW(root)
Input: root
Output: prints each level; returns the right side view
Initial call: PRINT-LEVELS-AND-RIGHT-VIEW(root)
1. queue <- [root]
2. rightView <- []
3. while queue is not empty
4.     levelSize <- size of queue
5.     for i <- 1 to levelSize
6.         node <- remove front of queue
7.         print node.val
8.         if i = levelSize
9.             add node.val to rightView
10.        if node.left != NULL
11.            add node.left to back of queue
12.        if node.right != NULL
13.            add node.right to back of queue
14.    print newline
15. return rightView
```

(b) Worked Example:
Level 0: queue=[5], levelSize=1. Print 5, add to rightView (last of level). Queue becomes [12,8].
Level 1: levelSize=2. Print 12 (not last), print 8 (last, add to rightView). Queue becomes [21,3,17,9].
Level 2: levelSize=4. Print 21,3,17 (not last), print 9 (last, add to rightView).

Levels: [5], [12,8], [21,3,17,9]. Right side view: 5, 8, 9, matching the expected output.
(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
1	queue ← [root]	1	1	1
2	rightView ← []	1	1	1
3	while test	1	$(h{+}1){+}1$	4
4	levelSize ← size of queue	1	$h{+}1$	3
5	for test (summed)	1	$n + (h{+}1)$	10
6	remove front	1	n	7
7	print node.val	1	n	7
8	if i=levelSize	1	n	7
9	add to rightView	1	$h{+}1$	3
10	check left	1	n	7
11	add left (real only)	1	depends on tree	3
12	check right	1	n	7
13	add right (real only)	1	depends on tree	3
14	print newline	1	$h{+}1$	3
15	return rightView	1	1	1

Dominant term $\rightarrow \Theta(n)$, since $h + 1$ is always much smaller than n for any real tree.

(d) Final Complexity:

Both cases $\Theta(n)$, you have to print and check every single node no matter what the tree looks like.

Twist: Your frequency count will depend on both n and the number of levels (h+1). Show where each term comes from.

Some of the lines depend on n and some depend on $h + 1$. The lines that deal with one node at a time (removing it, printing it, checking its kids) depend on n since there's n nodes total. But stuff that only happens once per level, like checking how big the level is or printing a new line, depends on $h + 1$ instead since there's only that many levels. Either way it ends up $\Theta(n)$ since n is always way bigger than $h + 1$.

Question 8: Zigzag (spiral) level order traversal

Input: the tree from arr
Expected output: 5, 8, 12, 21, 3, 17, 9
(a) Pseudocode:

```
ZIGZAG(root)
Input: root
Output: prints the zigzag level order sequence
Initial call: ZIGZAG(root)
1. queue <- [root]
2. level <- 0
3. while queue is not empty
4.      levelSize <- size of queue
5.      currentLevel <- []
6.      for i <- 1 to levelSize
7.          node <- remove front of queue
8.          add node.val to currentLevel
9.          if node.left != NULL
10.             add node.left to back of queue
11.         if node.right != NULL
12.             add node.right to back of queue
13.     if level is odd
14.         reverse currentLevel
15.     print currentLevel
16.     level <- level + 1
```

(b) Worked Example:
Level 0 (even): currentLevel equals [5], not reversed. Print 5.
Level 1 (odd): currentLevel equals [12, 8], reversed to [8, 12]. Print 8, 12.
Level 2 (even): currentLevel equals [21, 3, 17, 9], not reversed. Print 21, 3, 17, 9.

Final sequence: 5, 8, 12, 21, 3, 17, 9, which matches the expected output.
(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
1	queue ← [root]	1	1	1
2	level ← 0	1	1	1
3	while test	1	$(h{+}1){+}1$	4
4	levelSize ← size of queue	1	$h{+}1$	3
5	currentLevel ← []	1	$h{+}1$	3
6	for test (summed)	1	$n + (h{+}1)$	10
7	remove front	1	n	7
8	add to currentLevel	1	n	7
9	check left	1	n	7
10	add left (real only)	1	depends on tree	3
11	check right	1	n	7
12	add right (real only)	1	depends on tree	3
13	if level odd	1	$h{+}1$	3
14	reverse (only odd levels)	1	depends on tree	1
15	print currentLevel	1	$h{+}1$	3
16	level plus 1	1	$h{+}1$	3

Dominant term gives $\Theta(n)$, since $h + 1$ is always much smaller than n for any real tree.

(d) Final Complexity:

Best case is $\Theta(n)$. Worst case is $\Theta(n)$. No shortcut exists, every node has to get collected and every level has to get checked for odd or even no matter what.

Twist: Solve it with two stacks, or with a queue plus reversing every second level. Which uses less extra work in your frequency count?

Both ways end up $\Theta(n)$ overall. Doing it with a queue means you have to manually reverse every other level, which is a little bit of extra work each time. Using two stacks instead skips that step completely, since stacks just naturally come out backwards from how you put things in, so switching between two stacks gives you the zigzag automatically. So I guess the two stack way does a little less work, even though they're technically the same big-O.

Question 9: Is the tree complete?

Input: tree P from arr; tree Q given in level order with a gap: {5, 12, 8, NULL, 3, 17, 9} (12 has no left child)
Expected output: COMPLETE(P) = true; COMPLETE(Q) = false
(a) Pseudocode:

```
IS-COMPLETE(root)
Input: root
Output: boolean
Initial call: IS-COMPLETE(root)
1. queue <- [root]
2. seenNull <- FALSE
3. while queue is not empty
4.      node <- remove front of queue
5.      if node = NULL
6.          seenNull <- TRUE
7.      else
8.          if seenNull = TRUE
9.              return FALSE
10.         add node.left to back of queue
11.         add node.right to back of queue
12. return TRUE
```

(b) Worked Example:
Tree P: every NULL encountered is only ever followed by more NULLs, since all leaves have two NULL children and nothing real appears after them. The queue empties completely with seenNull becoming TRUE but never triggering line 9. Final output: TRUE.
Tree Q: when processing node 12, its missing left child sets seenNull to TRUE. Its right child, 3, is real and appears right after, so line 9 fires immediately. Final output: FALSE.

Both match the expected outputs.
(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
3	while test	1	$2n+2$	16
4	remove front	1	$2n+1$	15
5	if node=NULL	1	$2n+1$	15
6	seenNull $\leftarrow$ TRUE	1	$n+1$	8
8	if seenNull=TRUE	1	n	7
9	return FALSE	1	0	0
10	add left to queue	1	n	7
11	add right to queue	1	n	7
12	return TRUE	1	1	1

$T(n) \to \Theta(n)$
(d) Final Complexity:

Best case can be $\Theta(1)$, like if the very root is missing a left kid but has a right one, you catch that right away. Worst case is $\Theta(n)$ if the tree really is complete, since then you have to check everything before you can say yes.
Twist: Explain why INSERT and DELETE from the lecture always keep a tree complete.

Honestly I just thought about it logically, INSERT always looks for the first open spot starting from the left, going level by level, so it can't really leave a gap since it fills things in order. DELETE is kind of the opposite, it always takes from the very end, so even though it's removing stuff it's still not leaving a hole anywhere in the middle.

Question 10: Lowest common ancestor

Input: the tree from arr
Expected output: LCA(21, 3) = 12; LCA(21, 9) = 5; LCA(17, 8) = 8
(a) Pseudocode:

```
LCA(node, a, b)
Input: node, a, b
Output: node (the LCA, or NULL if neither found here)
Initial call: LCA(root, a, b)
1. if node = NULL
2.     return NULL
3. if node.val = a or node.val = b
4.     return node
5. leftResult <- LCA(node.left, a, b)
6. rightResult <- LCA(node.right, a, b)
7. if leftResult != NULL and rightResult != NULL
8.     return node
9. if leftResult != NULL
10.    return leftResult
11. return rightResult
```

(b) Worked Example:
For 21 and 3, they end up on different sides of node 12, so 12 itself becomes the answer. For 21 and 9, they're on completely opposite sides of the whole tree, so neither 12 nor 8 can answer it alone, it has to go all the way up to the root. For 17 and 8, node 8 is actually one of the two things I'm looking for, so it just returns itself right away without even needing to check if 17 is really underneath it.

All three match the expected outputs.
(c) Frequency Count:
Using LCA(21,9) as the worked example, since both targets are positioned on opposite sides of the tree, requiring the search to explore most of it before combining results.

Line	Operation	Cost	Frequency	Count
1	if node=NULL	1	depends on tree	11
2	return NULL	1	depends on tree	4
3	if node.val=a or b	1	depends on tree	7
4	return node (direct match)	1	depends on tree	2
5	leftResult ← LCA(...)	1	depends on tree	5
6	rightResult ← LCA(...)	1	depends on tree	5
7	if both non-NULL	1	depends on tree	5
8	return node (both found)	1	depends on tree	1
9	if leftResult != NULL	1	depends on tree	4
10	return leftResult	1	depends on tree	1
11	return rightResult	1	depends on tree	3

$T(n) \to \Theta(n)$ for this trace.

(d) Final Complexity:
Best case is $\Theta(1)$, if one target equals the root itself, or is found extremely early, returning almost immediately. Worst case is $\Theta(n)$, if both targets are deep in the tree on opposite sides, forcing exploration of most of the tree before the results can be combined, as shown in our traced example.
Twist: Solve it in one of two ways, finding both root to node paths and comparing them, or a single recursive pass. Compare the frequency counts of the two methods.

Doing one pass seems like less work than finding two separate paths, since each path basically means going through the whole tree once, so that's two full searches plus you still have to compare them at the end. Both end up n technically but the one pass way just skips doing stuff twice.

Question 11: Build a tree from its preorder and inorder traversals

Input: preorder = 5, 12, 21, 3, 8, 17, 9; inorder = 21, 12, 3, 5, 17, 8, 9
Expected output: the tree from arr; check: level order 5, 12, 8, 21, 3, 17, 9 and postorder 21, 3, 12, 17, 9, 8, 5
(a) Pseudocode:

```
BUILD-TREE(preorder, inorder, inStart, inEnd, preIndex)
Input: preorder array, inorder array, inStart, inEnd,
    preIndex
Output: node (root of the rebuilt subtree)
Initial call: BUILD-TREE(preorder, inorder, 0, n-1, preIndex
    =0)
1. if inStart > inEnd
2.     return NULL
3. rootVal <- preorder[preIndex]
4. preIndex <- preIndex + 1
5. rootIndex <- search for rootVal in inorder, between
    inStart and inEnd
6. node <- new node with value rootVal
7. node.left <- BUILD-TREE(preorder, inorder, inStart,
    rootIndex-1, preIndex)
8. node.right <- BUILD-TREE(preorder, inorder, rootIndex+1,
    inEnd, preIndex)
9. return node
```

(b) Worked Example:

Call	Root built	inStart, inEnd	rootIndex
1	5	0, 6	3
2 (left of 5)	12	0, 2	1
3 (left of 12)	21	0, 0	0
4 (right of 12)	3	2, 2	2
5 (right of 5)	8	4, 6	5
6 (left of 8)	17	4, 4	4
7 (right of 8)	9	6, 6	6

Resulting tree matches the original exactly. Level order 5, 12, 8, 21, 3, 17, 9 and postorder 21, 3, 12, 17, 9, 8, 5, both matching the expected check values.
(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
1	if inStart greater than inEnd	1	$2n+1$	15
2	return NULL	1	$n+1$	8
3	rootVal from preorder	1	n	7
4	preIndex plus 1	1	n	7
5	search for rootVal in inorder	1	depends on tree, bounded by n each call	12
6	create new node	1	n	7
7	build left subtree	1	n	7
8	build right subtree	1	n	7
9	return node	1	n	7

$T(n) = (2n+1) + (n+1) + n + n + (\text{search cost}) + n + n + n \to \Theta(n^2)$ worst case, since the search on line 5 can scan up to the full remaining range each time, adding up to the same triangular pattern seen in other problems.
$15 + 8 + 7 + 7 + 12 + 7 + 7 + 7 + 7 = 77$

(d) Final Complexity:

Best case is $\Theta(n)$. Worst case is $\Theta(n^2)$, when the tree is shaped so each search has to scan through most of what is left, like a chain.

Twist: If you search the inorder list for the root each time, what is the worst case frequency count? Show how a lookup table, mapping value to inorder index, reduces it to Theta of n.

Without a lookup table, each of the n calls might have to scan a big part of the remaining array to find the root, giving $\Theta(n^2)$ worst case.
If you build a lookup table once at the start, mapping each value to its position in inorder, then line 5 becomes a single lookup instead of a scan:

Line	Operation	Cost	Frequency	Count
setup	build lookup table	1	n	7
1	if inStart greater than inEnd	1	$2n+1$	15
2	return NULL	1	$n+1$	8
3	rootVal from preorder	1	n	7
4	preIndex plus 1	1	n	7
5	rootIndex from lookup table	1	n	7
6	create new node	1	n	7
7	build left subtree	1	n	7
8	build right subtree	1	n	7
9	return node	1	n	7

$T(n) \to \Theta(n)$ with the lookup table, compared to $\Theta(n^2)$ without it, since every search becomes a single lookup instead of a scan.

Question 12: Diameter of a binary tree

Input: the tree from arr
Expected output: 4 (for example the path 21, 12, 5, 8, 17)
(a) Pseudocode:

```
COMPUTE-HEIGHT(node, bestDiameter)
Input: node, bestDiameter (a shared variable tracking the
    best found so far)
Output: integer height of this subtree
Initial call: bestDiameter <- -1; COMPUTE-HEIGHT(root,
    bestDiameter)
1. if node = NULL
2.     return -1
3. leftHeight <- COMPUTE-HEIGHT(node.left, bestDiameter)
4. rightHeight <- COMPUTE-HEIGHT(node.right, bestDiameter)
5. localDiameter <- leftHeight + rightHeight + 2
6. if localDiameter > bestDiameter
7.     bestDiameter <- localDiameter
8. return 1 + max(leftHeight, rightHeight)
```

(b) Worked Example:

Node	leftHeight	rightHeight	localDiameter	New best
21	-1	-1	0	yes, bestDiameter = 0
3	-1	-1	0	no
12	0	0	2	yes, bestDiameter = 2
17	-1	-1	0	no
9	-1	-1	0	no
8	0	0	2	no
5	1	1	4	yes, bestDiameter = 4

Final diameter is 4, matching the expected output and the example path 21, 12, 5, 8, 17.

(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
1	if node=NULL	1	$2n+1$	15
2	return -1	1	$n+1$	8
3	leftHeight from left subtree	1	n	7
4	rightHeight from right subtree	1	n	7
5	localDiameter calculated	1	n	7
6	check against best so far	1	n	7
7	update best (only if greater)	1	depends on tree	3
8	return height	1	n	7

$T(n) = (2n+1) + (n+1) + n + n + n + n + n + n = 9n + 2 \rightarrow \Theta(n)$

(d) Final Complexity:

Best case is $\Theta(n)$. Worst case is $\Theta(n)$. No shortcut exists, every node has to get visited once to compute both height and diameter together no matter the shape.

Twist: A simple solution calls HEIGHT at every node and takes Theta of n squared on a chain. Give a solution that computes the height and the diameter in the same pass, and prove with your frequency count that it is Theta of n.

If you ran height separately at every node it would be really wasteful, you'd keep recalculating stuff you already did, especially bad on a chain since there's no splitting to make things smaller, that's where the n^2 comes from I think. But if you just do height and check the diameter thing at the same time in one function, you only touch each node once, so it's n instead, which the table kind of shows since everything only depends on n, nothing's getting repeated.

Question 13: Serialize and deserialize a binary tree

Input: the tree from arr
Expected output: 5 12 21 N N 3 N N 8 17 N N 9 N N (deserializing this list gives back the tree from arr)
(a) Pseudocode:

```
SERIALIZE(node, result)
Input: node, result (a shared list being built)
Output: nothing directly, result is filled as a side effect
Initial call: result <- []; SERIALIZE(root, result)
1. if node = NULL
2.     add "N" to result
3.     return
4. add node.val to result
5. SERIALIZE(node.left, result)
6. SERIALIZE(node.right, result)

DESERIALIZE(list, index)
Input: list, index (a shared position tracker)
Output: node (root of the rebuilt subtree)
Initial call: index <- 0; DESERIALIZE(list, index)
1. if list[index] = "N"
2.     index <- index + 1
3.     return NULL
4. node <- new node with value list[index]
5. index <- index + 1
6. node.left <- DESERIALIZE(list, index)
7. node.right <- DESERIALIZE(list, index)
8. return node
```

(b) Worked Example:
Serialize visits 5, 12, 21, N, N, 3, N, N, 8, 17, N, N, 9, N, N, writing N for every empty spot instead of skipping it, which gives the expected list.
Deserialize reads that same list back in the same order, using a counter that only moves forward when it actually uses a value. 5 becomes the root, 12 becomes its left side, 21 becomes 12's left side with no children of its own, and so on until the whole list is used and the tree matches the original.

Both procedures match the expected output exactly.
(c) Frequency Count:

	Line	Operation	Cost	Frequency	Count
SERIALIZE:	1	if node=NULL	1	$2n+1$	15
	2	add N to result	1	$n+1$	8
	3	return	1	$n+1$	8
	4	add node value	1	n	7
	5	recurse left	1	n	7
	6	recurse right	1	n	7

$T(n) = (2n+1) + (n+1) + (n+1) + n + n + n \to \Theta(n)$
$15 + 8 + 8 + 7 + 7 + 7 = 52$

	Line	Operation	Cost	Frequency	Count
	1	if list value is N	1	$2n+1$	15
	2	advance index	1	$n+1$	8
	3	return NULL	1	$n+1$	8
DESERIALIZE:	4	create node	1	n	7
	5	advance index	1	n	7
	6	recurse left	1	n	7
	7	recurse right	1	n	7
	8	return node	1	n	7

$T(n) = (2n+1) + (n+1) + (n+1) + n + n + n + n + n \to \Theta(n)$
$15 + 8 + 8 + 7 + 7 + 7 + 7 + 7 = 66$

(d) Final Complexity:

Best case is $\Theta(n)$. Worst case is $\Theta(n)$. Both procedures have to touch every node and every missing child marker exactly once no matter the shape.

Twist: Why does the list contain exactly 2n plus 1 items? Use this fact in the frequency count of both procedures.

The list has $2n+1$ things in it because it has n real nodes plus $n+1$ empty spots. Both functions basically touch every single one of those spots exactly once, so they're both $\Theta(n)$.

Question 14: Is the tree a valid binary search tree?

Input: tree P from arr; tree S given in level order: 9, 5, 17, 3, 8, 12, 21
Expected output: ISBST(P) = false; ISBST(S) = true
(a) Pseudocode:

```
IS-BST(node, low, high)
Input: node, low, high (the allowed range for this node's
    value)
Output: boolean
Initial call: IS-BST(root, -infinity, +infinity)
1. if node = NULL
2.     return TRUE
3. if node.val <= low or node.val >= high
4.     return FALSE
5. return IS-BST(node.left, low, node.val) and IS-BST(node.
    right, node.val, high)
```

(b) Worked Example:

	Call	Lines executed	Returns
Tree P:	IS-BST(12, -infinity, 5)	1,3,4	FALSE
	IS-BST(5, -infinity, +infinity)	1,3,5	FALSE

Final output: FALSE, matching the expected output.

	Call	Lines executed	Returns
Tree S:	IS-BST(3.left)=IS-BST(NULL)	1,2	TRUE
	IS-BST(3.right)=IS-BST(NULL)	1,2	TRUE
	IS-BST(3, -infinity, 5)	1,3,5	TRUE
	IS-BST(8.left)=IS-BST(NULL)	1,2	TRUE
	IS-BST(8.right)=IS-BST(NULL)	1,2	TRUE
	IS-BST(8, 5, 9)	1,3,5	TRUE
	IS-BST(5, -infinity, 9)	1,3,5	TRUE
	IS-BST(12.left)=IS-BST(NULL)	1,2	TRUE
	IS-BST(12.right)=IS-BST(NULL)	1,2	TRUE
	IS-BST(12, 9, 17)	1,3,5	TRUE
	IS-BST(21.left)=IS-BST(NULL)	1,2	TRUE
	IS-BST(21.right)=IS-BST(NULL)	1,2	TRUE
	IS-BST(21, 17, +infinity)	1,3,5	TRUE
	IS-BST(17, 9, +infinity)	1,3,5	TRUE
	IS-BST(9, -infinity, +infinity)	1,3,5	TRUE

Final output is TRUE, matching the expected output.
(c) Frequency Count:
Using tree S as the worked example, since it requires checking every node before confirming true.

Line	Operation	Cost	Frequency	Count
1	if node=NULL	1	$2n+1$	15
2	return TRUE	1	$n+1$	8
3	if value out of range	1	n	7
4	return FALSE	1	0	0
5	recursive combine	1	n	7

$T(n) \to \Theta(n)$

(d) Final Complexity:

Best case is $\Theta(1)$, if a violation exists near the root, caught within just a couple of calls. Worst case is $\Theta(n)$, if the tree genuinely is a valid binary search tree, every node must be checked before confirming true.

Twist: Give a second solution that uses an inorder traversal. Compare both frequency counts. What is the best case for each?

```
IS-BST-INORDER(node, prevVal)
Input: node, prevVal (a shared variable tracking the last
    value seen)
Output: boolean
Initial call: prevVal <- -infinity; IS-BST-INORDER(root,
    prevVal)
1. if node = NULL
2.     return TRUE
3. if not IS-BST-INORDER(node.left, prevVal)
4.     return FALSE
5. if node.val <= prevVal
6.     return FALSE
7. prevVal <- node.val
8. return IS-BST-INORDER(node.right, prevVal)
```

Line	Operation	Cost	Frequency	Count
1	if node=NULL	1	$2n+1$	15
2	return TRUE	1	$n+1$	8
3	recurse left, check result	1	n	7
4	return FALSE (only on violation)	1	depends on tree	0
5	if node.val <= prevVal	1	n	7
6	return FALSE (only on violation)	1	depends on tree	0
7	prevVal $\leftarrow$ node.val	1	n	7
8	recurse right	1	n	7

$T(n) \to \Theta(n)$

Both approaches give $\Theta(n)$ worst case. The range-passing solution has a best case of $\Theta(1)$, since a violation can be caught anywhere. This inorder solution has a best case of only $\Theta(h)$, since it must always travel to the leftmost leaf first before making its very first comparison.

Question 15: Vertical order traversal

Input: the tree from arr
Expected output: distance -2: 21, distance -1: 12, distance 0: 5, 3, 17, distance +1: 8, distance +2: 9
(a) Pseudocode:

```
VERTICAL-ORDER(root)
Input: root
Output: prints nodes grouped by horizontal distance, left to
    right
Initial call: VERTICAL-ORDER(root)
1. queue <- [(root, 0)]
2. columnMap <- empty map
3. while queue is not empty
4.      (node, dist) <- remove front of queue
5.      if dist not in columnMap
6.          columnMap[dist] <- []
7.      add node.val to columnMap[dist]
8.      if node.left != NULL
9.          add (node.left, dist-1) to back of queue
10.     if node.right != NULL
11.         add (node.right, dist+1) to back of queue
12. for each distance d, from smallest to largest
13.     print columnMap[d]
```

(b) Worked Example:

Node	Distance
21	-2
12	-1
5	0
3	0
17	0
8	+1
9	+2

Grouping by distance, from smallest to largest, matches the expected output exactly.
(c) Frequency Count:

Line	Operation	Cost	Frequency	Count
1	queue starts with root at distance 0	1	1	1
2	columnMap starts empty	1	1	1
3	while test	1	$n+2$	8
4	remove front	1	n	7
5	check if distance already in map	1	n	7
6	create new column entry	1	depends on tree	5
7	add node to its column	1	n	7
8	check left child	1	n	7
9	add left to queue, real only	1	depends on tree	3
10	check right child	1	n	7
11	add right to queue, real only	1	depends on tree	3
sort	sort the distinct distances before printing	1	up to $n \log n$	5
12	loop over sorted distances	1	number of columns	5
13	print columnMap[d]	1	number of columns	5

$T(n) \to \Theta(n \log n)$, since sorting the distances can cost more than n if there are many columns.

(d) Final Complexity:

Best case is $\Theta(n)$ Worst case is $\Theta(n \log n)$, since sorting the columns can cost more than just visiting every node.

Twist: Your queue must hold node and distance pairs, and you need a structure that groups nodes by distance. Include the cost of grouping and printing in the frequency count. How many columns can a tree of n nodes have?

The queue holds both the node and its distance together, and a map groups things by that distance as they come in. Adding something to a column doesn't cost much on its own, but sorting all the distinct distances before printing can cost up to $n \log n$, which is why the final answer is $\Theta(n \log n)$ instead of just $\Theta(n)$. The most columns you could ever have is n, since that would mean literally every single node ended up at its own distance, like if the tree zigzagged the whole way down.